

GROUP 25

Đặng Hoàng Sơn - 20235994

Bùi Doãn Khang - 20235950

Ngô Đức Huy - 20235946

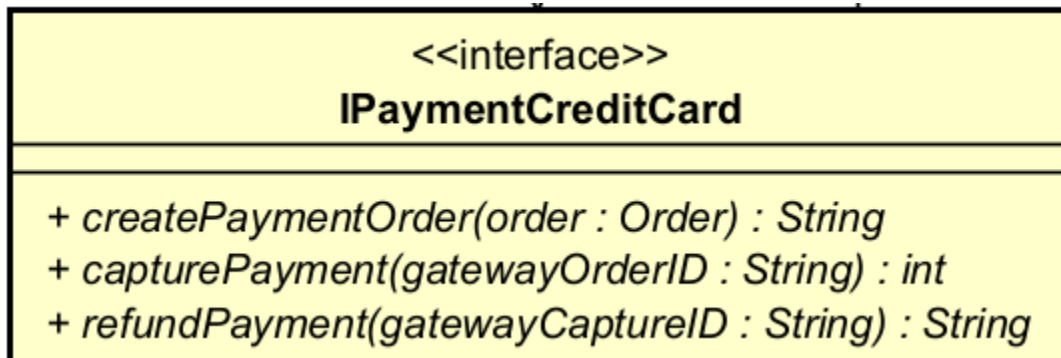
Lê Kế Đức - 20235912

Nguyễn Hoàng Trung Kiên – 20235960

DETAILED DESIGN: PAYPAL SUBSYSTEM

1. Interface

1.1 Design for interface "IPaymentCreditCard"



Description: This interface defines the standard set of operations for credit card payment processing within the AIMS system. It acts as a contract that any payment gateway subsystem (like PayPal) must realize, ensuring low coupling between the core system and external gateways.

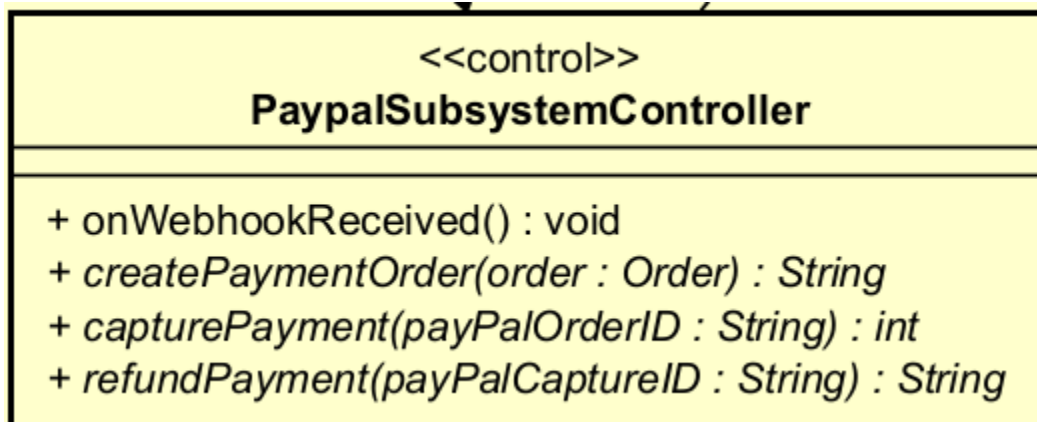
Table 1. Operation design

#	Name	Return type	Description (purpose)
1	createPaymentOrder(order: Order)	String	Initializes an order on the PayPal platform and returns the approval URL for the client.
2	capturePayment(payPalOrderID: String)	int	Executes the final fund transfer using

			the provided gateway order ID.
3	refundPayment(payPalCaptureID: String)	String	Processes the reversal of a successful transaction using the capture ID.

2. Controller

2.1 Design for class "PaypalSubsystemController"



Description: The central control class of the PayPal subsystem. It implements the IPaymentCreditCard interface and acts as a facade, orchestrating data flow between the AIMS system, the boundary classes, and the data entities.

Table 1. Operation design

#	Name	Return type	Description (purpose)
1	onWebhookReceived(eventPayload: String)	void	Listens for and processes asynchronous event notifications from PayPal.
2	createPaymentOrder(order: Order)	String	Generates the payload, requests token, and initiates the payment process.
3	capturePayment(payPalOrderID: String)	int	Sends a capture request for an approved order and updates the local entity.
4	refundPayment(payPalCaptureID: String)	String	Executes the refund process

			using the transaction's Capture ID.
--	--	--	-------------------------------------

Parameter:

- order: Order object containing all purchase details (e.g., items, total amount).
- paypalOrderID: A unique string identifier for the created order issued by PayPal.
- paypalCaptureID: A unique string identifier for the successfully captured fund, required for refunds.
- eventPayload: A JSON string containing the webhook event data sent asynchronously by PayPal.

Exception:

- PayPalAuthenticException: Thrown if OAuth2 authentication with PayPal fails.
- UserCancelledException: Thrown if the user aborts the payment on the PayPal approval page.
- InsufficientBalanceException: Thrown if the customer has inadequate funds or the card is rejected (applicable during both order creation and the final capturePayment phase).
- UnknownException: Thrown for any other unexpected HTTP errors, gateway-side crashes, or unclassified failures during the capture process.

Method:

- createPaymentOrder: Transforms the Order object into a request string, utilizes PayPalBoundary to communicate with the API, and parses the response to extract the approval URL.
- capturePayment: Invokes the boundary to capture funds. If PayPal declines the capture (e.g., card expired), it throws an InsufficientBalanceException or UnknownException. If successful, it calls captureUpdate on the PayPalTransaction entity to persist the final status.

Other diagrams:

- Sequence Diagram: Refer to SD CreateOrder and SD CaptureOrder for detailed execution flows.

3. Entity

3.1 Design for class "PaypalTransaction"

<<entity>> PaypalTransaction	
- payPalOrderID : String - status : String - amount : double - payPalCaptureID : String - payerID : int	
+ savePayPalTransaction() : void + captureUpdate(transactionDetails : String) : void	

Description: An entity class responsible for storing the persistent data of a specific PayPal transaction. It maintains the mapping between AIMS system data and external PayPal identifiers.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	payPalOrderID	String	null	The order identifier provided by PayPal API.
2	status	String	"CREATED"	Current state of the transaction.
3	amount	double	0.0	The total monetary value of the transaction.
4	payPalCaptureID	String	null	The capture identifier, populated after a successful capture.
5	payerID	int	0	The unique identifier of the user making the payment.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	savePayPalTransaction()	void	Persists the newly created transaction

			data into the database.
2	captureUpdate(transactionDetails: String)	void	Updates the status and Capture ID after a successful fund capture.

Parameter:

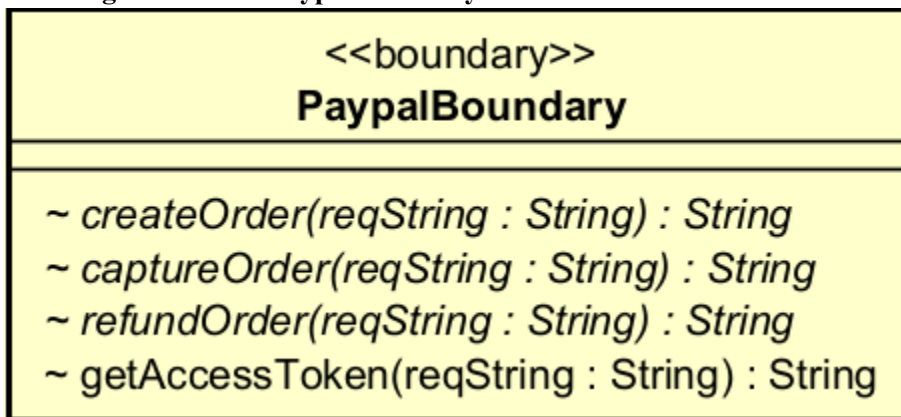
- transactionDetails: A JSON string or parsed object containing the final captured data returned by PayPal.

Method:

- captureUpdate: Parses the transactionDetails string to extract the final status (e.g., "COMPLETED") and the capture ID, then overwrites the entity's current attributes.

4. Package: Boundary

4.1 Design for class "PaypalBoundary"



Description: A boundary class that provides a low-level interface to the external PayPal REST API. It handles HTTP POST/GET requests and manages network I/O.

Table 1. Operation design

#	Name	Return type	Description (purpose)
1	createOrder(reqString: String)	String	Submits an HTTP POST request to create an order.
2	captureOrder(reqString: String)	String	Submits an HTTP POST request to capture payment.
3	refundOrder(reqString: String)	String	Submits an HTTP POST request to refund a captured

			payment.
4	getAccessToken(reqString: String)	String	Requests an OAuth2 Bearer token using Client ID and Secret.

Parameter:

- reqString: A meticulously formatted JSON string containing the required payload and headers (including the Authorization Bearer Token) for the specific PayPal endpoint.

Method:

- Converts reqString into an HTTP request payload, opens a secure HTTPS connection to the appropriate PayPal endpoint (Live or Sandbox), and returns the raw JSON response body as a String.

5. Request & Response Handler

5.1 Design for class "PayPalAccessTokenReq"

PayPalAccessTokenReq
- clientID : String - secretKey : String
~ buildReqString() : String

Description: A class responsible for encapsulating the client credentials required to request an OAuth2 Access Token from the PayPal API.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	clientID	String	null	The public client identifier provided by the PayPal Developer Dashboard.
2	secretKey	String	null	The private secret key used for OAuth2 Basic Authentication.

Table 2. Operation design

#	Name	Return type	Description
---	------	-------------	-------------

			(purpose)
1	buildReqString()	String	Converts credentials into the required Base64-encoded Basic Auth string format.

5.2 Design for class "PayPalAccessTokenRes"

PayPalAccessTokenRes
- accessToken : String - expiresIn : int
~ parseResString() : void

Description: A class responsible for parsing the authentication response from PayPal's OAuth2 API. It extracts the raw access token string and its validity duration (Time To Live).

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	accessToken	String	null	The extracted OAuth2 Bearer token used to authenticate subsequent API calls.
2	expiresIn	int	0	The lifespan of the token in seconds (typically 32400 seconds / 9 hours).

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	parseResString(resString: String)	void	Parses the raw JSON response to extract the token and its TTL.

Parameter:

- resString: A JSON string returned by the PayPalBoundary after successfully calling the PayPal OAuth2 token endpoint.

5.3 Design for class "PayPalOrderReq"

PayPalOrderReq
- accessToken : String - amount : double - purchaseUnits : OrderItem[]
~ buildReqString() : String

Description: A class responsible for encapsulating the necessary data to request the creation of a new PayPal order.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	accessToken	String	null	The valid OAuth2 Bearer token retrieved from Redis or PayPal API.
2	amount	double	0.0	The total monetary value of the order to be paid.
3	purchaseUnits	OrderItem[]	null	An array containing the specific details of the order items.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	buildReqString()	String	Converts the object's attributes into a strictly formatted JSON payload.

5.4 Design for class "PayPalOrderRes"

PayPalOrderRes
- PayPalOrderID : String - status : String
~ parseResString() : void

Description: A class designed to parse and hold the data returned by the PayPal API after a successful order creation request.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	paypalOrderID	String	null	The unique ID generated by PayPal for the newly created order.
2	status	String	null	The initial status of the order (typically "CREATED").

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	parseResString(resString: String)	void	Parses the raw JSON response to extract the order ID and approval URL.

5.5 Design for class "PayPalCaptureReq"

PayPalCaptureReq
- PayPalOrderID : String - accessToken : String
~ buildReqString() : String

Description: A class responsible for encapsulating the required parameters to execute a payment capture on a previously approved PayPal order.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	paypalOrderID	String	null	The unique identifier of the approved order to be captured.
2	accessToken	String	null	The valid OAuth2 Bearer token used for API authorization.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	buildReqString()	String	Constructs the JSON payload or formatting string for the capture request.

5.6 Design for class "PayPalCaptureRes"

PayPalCaptureRes
- PayPalCaptureID : String - status : String - paymentTime : timestamp
~ parseResString() : void

Description: A class that parses the JSON response returned by the PayPal API after a capture request. It extracts critical transaction details.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	paypalCaptureID	String	null	The unique identifier for the captured payment, strictly required for future refunds.
2	status	String	null	The final status of the capture process (e.g., "COMPLETED" or

				"DECLINED").
3	paymentTime	timestamp	null	The exact time when the payment was successfully captured.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	parseResString(resString: String)	void	Parses the raw JSON response to extract the capture ID, status, and payment time.

5.7 Design for class "PayPalRefundReq"

PayPalRefundReq
- PayPalCaptureID : String - accessToken : String
~ buildReqString() : String

Description: A class responsible for encapsulating the necessary parameters to request a refund for a previously captured PayPal transaction.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	paypalCaptureID	String	null	The unique identifier of the captured payment to be refunded.
2	accessToken	String	null	The valid OAuth2 Bearer token used for API authorization.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	buildReqString()	String	Constructs the JSON payload or formatting string for the refund

			request.
--	--	--	----------

5.8 Design for class "PayPalRefundRes"

PayPalRefundRes
- status : String
~ parseResString() : void

Description: A class that parses the JSON response returned by the PayPal API after a refund request.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	status	String	null	The status of the refund (typically "COMPLETED" or "PENDING").

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	parseResString(resString: String)	void	Parses the raw JSON response to extract the refund status.

6. Package: Service

6.1 Design for class "RedisService"

<<service>> RedisService
- host : String - port : int - client : RedisClient
+ getPayPalToken(key : String) : String + setPayPalToken(key : String, ttl : int) : void + deletePayPalToken(key : String) : void

Description: An infrastructure service class responsible for managing the caching mechanism of the system. In the context of the PayPal subsystem, it temporarily stores the OAuth2 Access Token to minimize redundant API calls.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	host	String	"localhost"	The hostname or IP address of the Redis server.
2	port	int	6379	The port number the Redis server is listening on.
3	client	RedisClient	null	The underlying library client instance used to execute Redis commands.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getPayPalToken(key: String)	String	Retrieves the stored access token from the cache using the specified key.
2	setPayPalToken(key: String, ttl: int)	void	Stores the access token in the cache with a specified Time-To-Live.
3	deletePayPalToken(key: String)	void	Manually evicts the

	String)		token from the cache before its expiration.
--	---------	--	---

Parameter:

- key: A unique string identifier used to store and retrieve the token (e.g., "paypal_access_token").
- ttl: The Time-To-Live value in seconds, representing how long the token should remain in the cache before automatically expiring.

Method:

- setPayPalToken: Utilizes the Redis SETEX (Set with Expiration) command to simultaneously store the token string and apply the ttl countdown.
- getPayPalToken: Executes a standard Redis GET command. If the key has expired, it automatically returns null.